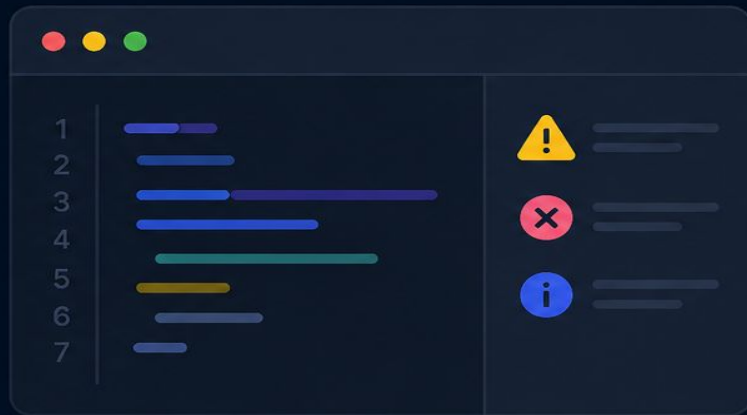


1. Зачем нужен ESLint

Зачем нужен ESLint

- ✓ **TypeScript** проверяет типы.
- ✓ **ESLint** проверяет качество кода.
- ✓ **ESLint** помогает находить лишние переменные, неиспользуемые импорты и простые ошибки.
- ✓ **Prettier** отдельно отвечает за форматирование, его сейчас не трогаем.



2. Установка

Установка

Запускаем:

```
> npm install -D eslint @eslint/js typescript-eslint
```

Говорим:



eslint

eslint — сам линтер.

@eslint/js

@eslint/js — базовые правила для JavaScript.



typescript-eslint

typescript-eslint — поддержка TypeScript.



Что установится:

- ✓ eslint — сам линтер
- ✓ @eslint/js — базовые правила для JavaScript
- ✓ typescript-eslint — поддержка TypeScript

3. Конфиг

Конфигурация ESLint

Создаем файл `eslint.config.js` в корне проекта



Игнорируем лишнее

Папки `dist` и `node_modules` не содержат исходный код, поэтому их проверять не нужно.



Базовые правила JavaScript

`js.configs.recommended` включает набор проверенных базовых правил для JS.



Правила для TypeScript

`tseslint.configs.recommended` добавляет правила, учитывающие особенности TypeScript.



Важно: порядок имеет значение.

Сначала базовые правила JavaScript, затем правила для TypeScript.

```
eslint.config.js

1  import js from "@eslint/js";
2  import tseslint from "typescript-eslint";
3
4  export default tseslint.config(
5    {
6      ignores: ["dist", "node_modules"],
7    },
8    js.configs.recommended,
9    tseslint.configs.recommended,
10 );
```



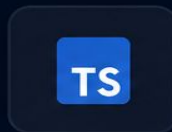
@eslint/js
(базовые правила)

+



js.configs.recommended
(для JavaScript)

+



typescript-eslint
(поддержка TS)

+



tseslint.configs.recommended
(правила для TypeScript)



`eslint.config.js`

ваша конфигурация ESLint

4. Скрипт

Скрипт для ESLint

Открываем `package.json` и добавляем скрипт.

1

`package.json`

Открываем файл

Находим секцию `scripts`.

2

`</>`

Добавляем скрипт

Добавляем команду `lint`.

3

`>_`

Сохраняем и используем

Теперь можно запускать `npm run lint`.

Структура проекта

```
├─ my-app
├─ src
│  ├─ App.tsx
│  └─ index.ts
├─ eslint.config.js
├─ package.json
└─ ...
```



Важно

Скрипт проверяет только исходные файлы проекта и не затрагивает сторонние библиотеки и сборку.



`package.json`

```
1 {
2   "name": "my-app",
3   "version": "1.0.0",
4   "private": true,
5   "scripts": {
6     "lint": "eslint \"src/**/*.{ts,tsx}\""
7   },
8   "dependencies": {
9     ...
10  }
```



«`src`» — проверяем только файлы в папке `src`.



«`*.{ts,tsx}`» — проверяем файлы с расширениями `.ts` и `.tsx`.



Запуск: `npm run lint`

Команда запустит ESLint для всех файлов `.ts` и `.tsx` внутри `src`.

5. Первый запуск

Первый запуск ESLint

Запускаем команду:

```
$ npm run lint
```



Что происходит:



Если ошибок нет

Команда завершится спокойно, и вы увидите сообщение об успешной проверке.



Если ошибки есть

ESLint покажет файл, строку, правило и описание проблемы.



Почему это важно

Теперь у нас есть быстрый способ проверять качество кода в проекте.

Пример вывода:

```
Terminal

$ npm run lint

src/components/Button.tsx
 12:7  error  'unusedValue' is assigned a value but never used
      @typescript-eslint/no-unused-vars

✖ 1 problem (1 error, 0 warnings)
   0 errors and 0 warnings potentially fixable with the `--fix` option.

npm ERR! code 1
npm ERR! path /Users/user/my-app
npm ERR! command failed
```

Что означает вывод:

src/components/Button.tsx	Файл, в котором найдена проблема.
12:7	Строка 12, символ 7.
error	Уровень проблемы (error, warning).
no-unused-vars	Правило ESLint, которое сработало.
✖ 1 problem	Итог: найдено 1 проблема.

6. Демонстрация ошибки

Демонстрация ошибки ESLint

Добавим временный код с ошибкой и посмотрим, как ESLint её найдет.

1 Добавляем код с ошибкой

src/components/Button.tsx

```
1 import React from 'react';
2
3 const Button = () => {
4   const unusedValue = 123;
5   return <button>Click me</button>;
6 };
7
```

← неиспользуемая
переменная

2 Запускаем линтер

```
$ npm run lint
```

3 Удаляем лишний код и запускаем снова

```
const unusedValue = 123;
```



npm run lint
Готово! Ошибок нет.

Что покажет ESLint

```
$ npm run lint
> my-app@1.0.0 lint
> eslint "src/**/*.{ts,tsx}"

src/components/Button.tsx
4:7  error  'unusedValue' is assigned a value but never used
@typescript-eslint/no-unused-vars
4:7  warning 'unusedValue' is assigned a value but never used
@typescript-eslint/no-unused-vars

✖ 1 problem (1 error, 0 warnings)
0 errors and 0 warnings potentially fixable with the `--fix` option.
```



Что произошло?

- ESLint нашел неиспользуемую переменную `unusedValue`.
- Правило `@typescript-eslint/no-unused-vars` запрещает неиспользуемые переменные.
- Удаляем лишний код и запускаем линтер снова — ошибок больше нет.



Зачем это нужно?

Такие проверки помогают поддерживать код чистым, понятным и безопасным.



Находит
лишний код



Предотвращает
ошибки



Делает код
чище

7. Проверка проекта

Проверка проекта

Проверяем код на соответствие правилам **ESLint** и **собираем** проект.

1 Запускаем линтинг

```
$ npm run lint
```



Если ошибок нет:

команда **завершится** спокойно.
Код соответствует правилам ESLint.



Если ошибки есть:

ESLint покажет файл, строку и правило.
Исправляем ошибки и запускаем снова.



Совет

Запускайте линтер перед каждой сборкой — это помогает поддерживать код чистым и единообразным.

2 Запускаем сборку

```
$ npm run build
```



Если сборка прошла успешно:

приложение собрано и готово к запуску.

Пример вывода

Линтинг (успешно)

```
$ npm run lint
> my-app@1.0.0 lint
> eslint "src/**/*.{ts,tsx}"
```

✓ No problems found.

Сборка (успешно)

```
$ npm run build
> my-app@1.0.0 build
> tsc -b && vite build

vite v5.2.0 building for production...
✓ 142 modules transformed.
dist/index.html    0.46 kB
dist/assets/index-abc123.js 125.34 kB
✓ built in 1.23s
```



Итог



lint — проверяет
правила кода



build — проверяет
сборку приложения



**Чистый код + успешная сборка =
стабильный проект**